

Assignment 17

registration_service.py

```
import re

class InvalidEmailError(ValueError):
    """Raised when the email is invalid."""
    pass

class UnderageError(RuntimeError):
    """Raised when the user's age is below 18."""
    pass

class RegistrationService:

    EMAIL_PATTERN = r'^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'

    def register_user(self, email: str, age: int) -> bool:

        # Internal invariant check
        assert email is not None, "System invariant violated: email cannot be None"

        # Validate email
        if not email or email.strip() == "":
            raise InvalidEmailError("Email cannot be empty.")

        if not re.match(self.EMAIL_PATTERN, email):
            raise InvalidEmailError("Invalid email format.")

        # Validate age
        if age < 18:
            raise UnderageError("User must be at least 18 years old.")

        return True
```

test_registration_service.py

```
import pytest
from registration_service import (
    RegistrationService,
    InvalidEmailError,
    UnderageError
)
```

```
@pytest.fixture
def service():
    return RegistrationService()
```

```
def test_successful_registration(service):
    assert service.register_user("user@example.com", 22) is True
```

```
def test_empty_email(service):
    with pytest.raises(InvalidEmailError):
        service.register_user("", 25)
```

```
def test_invalid_email_format(service):
    with pytest.raises(InvalidEmailError):
        service.register_user("invalid-email", 25)
```

```
def test_underage_user(service):
    with pytest.raises(UnderageError):
        service.register_user("user@example.com", 16)
```

```
def test_none_email_assertion(service):
    with pytest.raises(AssertionError):
        service.register_user(None, 20)
```